UPSA
UNIVERSITY OF PROFESSIONAL STUDIES, ACCRA
Scholarship with Professionalism

BSIT 401 Network and Systems Administration

Chapter 2: COMPUTING SYSTEM AND COMPONENTS

Lecturers

Dr Samuel Attuquayefio | Mr. Adams Azameti

CHAPTER 2 — COMPUTING SYSTEM AND COMPONENTS

1.0 Week 2 Overview

Week 2 examines the core components of a computing system from the viewpoint of a network and systems administrator. The emphasis is not merely on identifying hardware, but on understanding how the operating system exposes CPU, memory, storage, file systems, processes and services as manageable resources. Students learn how to inspect these resources, recognize normal and abnormal behaviour, and collect evidence that can support administrative decisions and troubleshooting.

CHAPTER 2 — COMPUTING SYSTEM AND COMPONENTS

1.1 Learning Outcomes

- Explain the major components of a modern computing system and their administrative significance.
- Interpret CPU architecture and processor information obtained from Linux administrative commands.
- Explain physical memory, virtual memory, cache and swap, and identify indicators of memory pressure.
- Distinguish disks, partitions, file systems, mount points and logical storage concepts.
- Inspect storage devices, file-system types, capacity and utilization using standard Linux tools.
- Explain processes, process states, process identifiers and resource consumption.
- Differentiate a process from a service and explain how services support organizational computing.
- Use a systematic evidence-based approach to identify likely CPU, memory or storage bottlenecks.
- Document command output and translate raw system information into an administrator-level interpretation.

WEEK 2: COMPUTING SYSTEM AND COMPONENTS

LECTURE NOTE
Level 400 | Credit Hours: 3

1. WEEK OVERVIEW

This week reviews the main components of a computing system — CPU, memory, storage, file systems, processes and services — from an administrator's perspective, with emphasis on how to inspect, monitor and manage system resources.

2. LEARNING OUTCOMES

By the end of this week, you should be able to:

1. Describe the main components of a computing system and their functions.
2. Inspect and monitor CPU, memory, storage, file systems, processes and services.
3. Use common Linux commands to obtain and interpret system information.
4. Manage system resources and identify performance bottlenecks.
5. Apply administrative knowledge to solve practical problems.

3. A COMPUTING SYSTEM: AN OVERVIEW

A computing system is a combination of hardware and software that works together to process data, store information and provide services to users and applications.

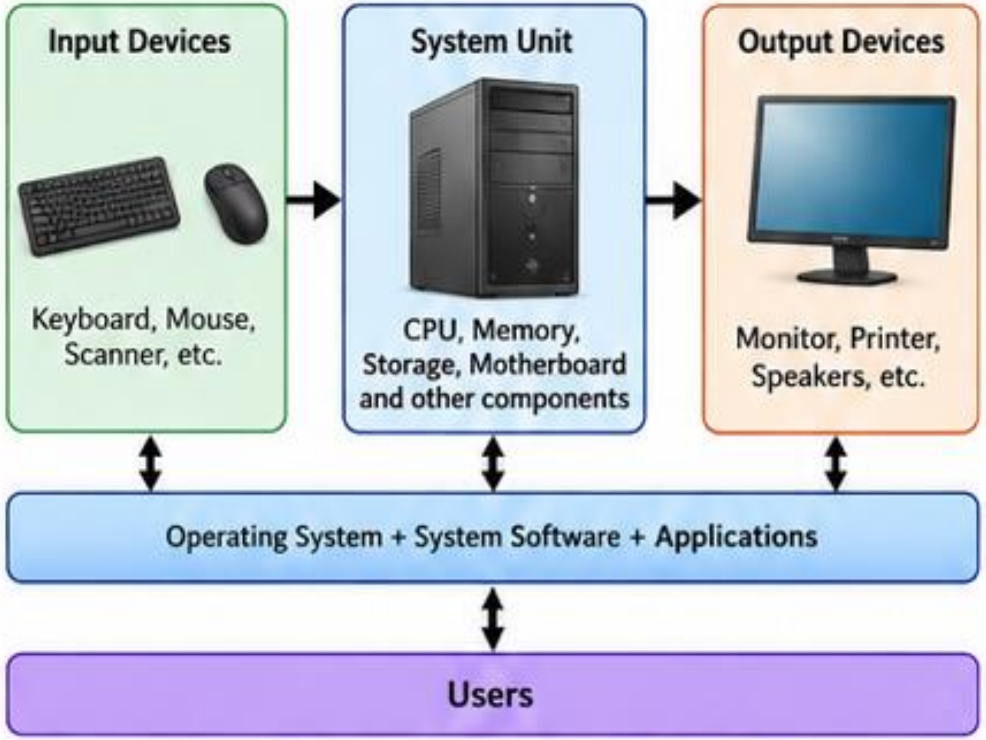


Figure 1: Basic structure of a computing system

4. MAJOR COMPONENTS AND THEIR ROLES

Component	Function	Examples
CPU (Processor)	Executes instructions, performs computations.	Intel Core i5, AMD Ryzen, ARM processors
Memory (RAM)	Temporarily stores data and instructions for active processes.	DDR4, DDR5
Storage	Permanently stores operating system, applications and user data.	HDD, SSD, NVMe
Motherboard	Connects all components and enables communication.	ATX, micro-ATX
Input/Output Devices	Interface with the user and external systems.	Keyboard, mouse, monitor, network card
Operating System	Manages hardware resources and provides services for applications.	Linux, Windows, macOS



CPU (Processor) **RAM (Memory)** **HDD (Storage)** **SSD (Storage)**

Figure 2: Examples of key hardware components

5. CPU: CONCEPTS AND MONITORING

- The CPU (Central Processing Unit) fetches, decodes and executes instructions.
- Key metrics: number of cores, threads, clock speed, CPU usage and load average.
- As an administrator, you monitor CPU utilisation to detect performance issues.

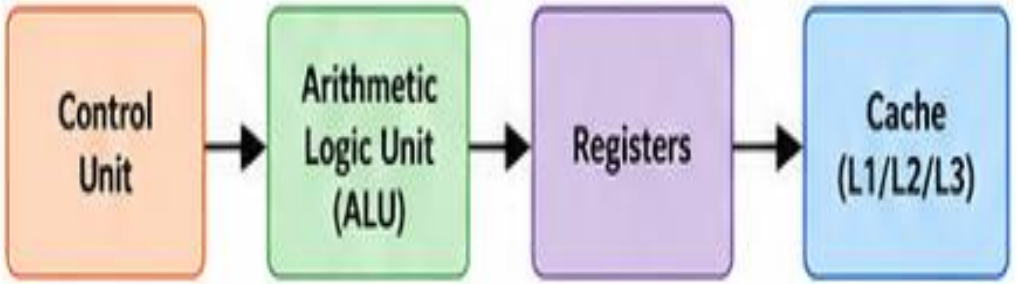


Figure 3: Simplified CPU architecture

6. MEMORY: RAM AND VIRTUAL MEMORY

- RAM stores data and instructions for active processes.
- When RAM is full, the system uses swap space on disk (virtual memory).
- Key metrics: total, used, free, available, swap usage.
- Monitoring memory helps you identify memory leaks and decide when to add more RAM.

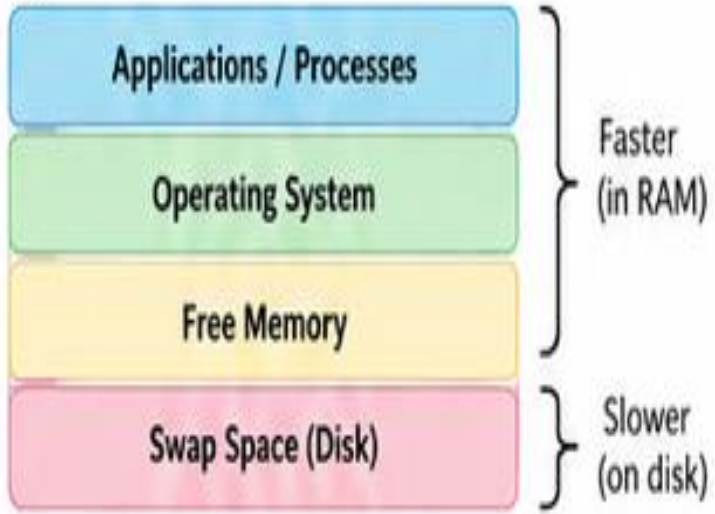


Figure 4: Memory layout in a Linux system

7. STORAGE AND FILE SYSTEMS

- Storage devices hold the operating system, applications and user data.
- Linux supports various file systems (ext4, xfs, btrfs, etc.).
- Key concepts: partitions, file systems, mount points, disk usage and inode usage.
- Administrators monitor disk space to prevent service interruption.

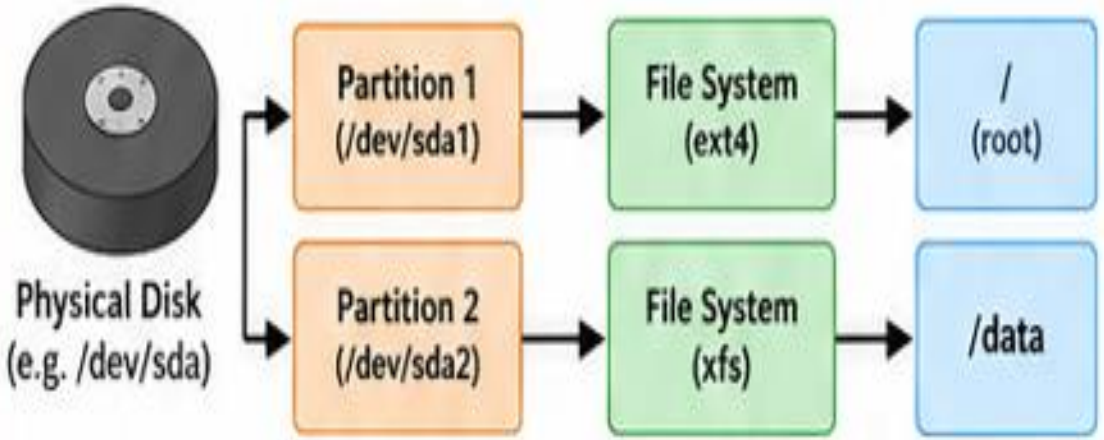


Figure 5: Disk, partitions, file systems and mount points

8. PROCESSES AND SERVICES

- A process is a running instance of a program.
- A service (daemon) is a background process that provides a specific function.
- As an administrator, you monitor, manage, start, stop and troubleshoot processes and services.

```
user@linux:~$ ps -ef | head
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 10:00 ?        00:00:05 /sbin/init
root           2       0  0 10:00 ?        00:00:00 [kthreadd]
root           3       2  0 10:00 ?        00:00:00 [rcu_gp]
root           4       2  0 10:00 ?        00:00:00 [rcu_par_gp]
root           5       2  0 10:00 ?        00:00:00 [kworker/0:0H-kblockd]
```

Figure 6: Example output of `ps -ef` command

UPSA
COMMUNICATION STUDIES

CHAPTER 2 — COMPUTING SYSTEM AND COMPONENTS

1.2 Key Terminology

Term	Administrator-oriented meaning
CPU	Central Processing Unit; executes machine instructions and coordinates computation.
Core	An independent processing unit within a CPU package.
Thread	A schedulable execution stream; hardware threads may be exposed through simultaneous multithreading.
RAM	Volatile main memory used by active programs and the operating system.
Swap	Disk- or partition-backed space used to support virtual memory when RAM pressure exists.
Block device	A storage device accessed in fixed-size blocks, such as an SSD or HDD.

1.2 Key Terminology (Cont.)

Term	Administrator-oriented meaning
Partition	A logical division of a physical storage device.
File system	A structure and set of rules used to organize and retrieve files on a storage volume.
Mount point	A directory at which a file system becomes accessible in the Linux directory tree.
Process	A running instance of a program with a PID and allocated resources.
Service	A long-running system process or group of processes that provides a capability, commonly managed by systemd on Linux.
Load average	A measure of runnable and uninterruptible work averaged over recent time periods.

2.0 Computing System Architecture from an Administrator's Perspective

A computing system can be viewed as a layered environment. Hardware provides processing, memory, storage and I/O capabilities.

The operating system abstracts and controls these resources. Processes and services use operating-system interfaces to deliver applications and network functions.

An administrator works across all layers because a failure in one layer can appear as a symptom in another.

Applications and User Services: **Web servers, databases, file services, business applications**
Processes and Services: **Running programs, daemons, scheduled work**
Operating System / Kernel: **Scheduling, memory management, file systems, device drivers, networking**
Hardware Resources: **CPU, RAM, storage, buses, NICs and other devices**
Physical Infrastructure: **Power, cooling, chassis and cabling**

Administrative reasoning should follow the dependency chain. For example, a web application that is slow may be caused by application code, a saturated database process, memory pressure, excessive disk I/O, CPU contention, or a broader infrastructure issue. Good administrators collect evidence before changing the system.

3.0 Central Processing Unit (CPU)

3.1 What the CPU Does

The CPU fetches, decodes and executes instructions. It performs arithmetic and logical operations, moves data, handles control flow and interacts with memory and I/O through the system architecture.

From an administrator's viewpoint, the important questions are: How many processing units are available? What architecture is present? Is the system virtualized? Is CPU demand exceeding capacity? Which processes are consuming CPU time?

3.2 CPU Concepts Administrators Must Understand

- **Architecture (for example x86-64 or ARM64):** affects binary compatibility and package selection.
- **Sockets:** physical CPU packages installed in a system.
- **Cores per socket:** physical execution cores available within each CPU.
- **Hardware threads / logical CPUs:** schedulable CPU contexts exposed to the operating system.
- **Clock frequency:** an indicator of operating speed, but not a complete measure of performance.
- **CPU cache:** fast memory close to the processor that reduces access latency to frequently used data.
- **Virtualization flags and hypervisor information:** important when systems run as virtual machines.

```
$ lscpu
$ nproc
```

Interpretation: ***lscpu*** summarizes CPU architecture, logical CPU count, cores, sockets, NUMA information and virtualization-related attributes.

nproc reports the number of processing units currently available to the process, which may differ from the host total in constrained environments such as containers or CPU-limited workloads.

Observation	Possible meaning	Administrator response
High %us (user CPU)	User-space applications are actively consuming CPU	Identify top CPU consumers; inspect workload and concurrency.
High %sy (system CPU)	Kernel activity is relatively high	Investigate networking, storage, drivers or syscall-heavy workloads.
High load with CPU near 100%	Runnable work exceeds available CPU capacity	Compare load with logical CPU count; identify processes and workload spikes.
High load but CPU not fully busy	Tasks may be blocked in I/O or uninterruptible waits	Inspect disk and service behavior; do not assume CPU shortage.
One process dominates	A single application may be the bottleneck	Validate whether usage is expected; inspect process configuration and logs.

4.0 Memory Management

4.1 Physical and Virtual Memory

RAM stores code and data needed by active processes and by the kernel. Modern operating systems also use free memory for caches because unused RAM provides little benefit.

Therefore, a low “free” value alone does not automatically mean a memory problem. Administrators should examine available memory, swap activity, application working sets and system responsiveness together.

```
$ free -h
$ cat /proc/meminfo | head
$ swapon --show
```

Field / Concept	Meaning	Administrative interpretation
total	Installed/usable memory reported to the OS	Baseline capacity.
used	Memory considered in active use	Must be interpreted together with cache/buffers.
free	Completely unused RAM	Low free memory is not by itself proof of pressure.
buff/cache	Memory used for kernel buffers and file cache	Often reclaimable when applications need memory.
available	Estimate of memory available for starting new work without heavy swapping	Usually a better quick indicator than free memory.
swap	Disk-backed virtual-memory space	Persistent growth or active paging may indicate memory pressure.

Important interpretation rule

Do not diagnose “low RAM” from a single number.

Combine free -h with process memory usage, swap status, workload behaviour and system responsiveness.

Evidence-based diagnosis is a core systems-administration habit.

5.0 Storage Devices and Storage Hierarchy

Persistent storage holds operating-system files, applications, logs, user data and configuration.

Administrators must distinguish the physical device from its partitions and from the file systems mounted on those partitions.

A server can have free space on one volume while another critical file system is completely full.

Technology	Storage mechanism	Administrative significance
HDD	Magnetic rotating media	High capacity, lower cost per GB, higher access latency, moving parts.
SATA SSD	Flash storage via SATA	Lower latency and faster random I/O than HDD; no moving parts.
NVMe SSD	Flash storage over PCIe/NVMe	Very low latency and high throughput; common in high-performance systems.

5.2 Inspecting Block Devices

```
$ lsblk -f
```

Typical information includes device name, file-system type, UUID, label, size-related context and mount point.

Use it to answer questions such as:

- Which file system is mounted as root (/)?
- Which device holds /home?
- Is a removable device present?
- Are there unmounted partitions?

6.0 File Systems and Mounting

A file system defines how files and directories are represented on a storage volume. Linux integrates multiple file systems into a single directory tree.

A mounted file system becomes accessible at a mount point. This means administrators must understand both capacity and where that capacity is attached within the directory tree.

File system	Typical role	Administrative note
ext4	General-purpose Linux file system	Common default; journaling; widely supported.
XFS	High-performance journaling file system	Common on enterprise Linux; strong scalability for large filesystems.
Btrfs	Copy-on-write file system	Snapshots and advanced storage features; deployment varies.
vfat/exFAT	Cross-platform removable-media formats	Useful for USB/removable media; different permission semantics.
tmpfs	Memory-backed temporary file system	Fast and volatile; consumes memory and disappears on reboot.

```
$ df -hT
$ findmnt
```

df -hT reports mounted file-system capacity, used space, available space and file-system type.

findmnt shows the mount hierarchy and makes it easier to understand which source device or pseudo-file-system is attached to each target directory.

6.2 Disk Capacity versus File-System Capacity

A physical disk can contain several partitions or logical volumes. Each can host a separate file system. Therefore “the disk has 500 GB” does not imply that every directory has access to all 500 GB. The capacity available to a directory depends on the file system on which that directory resides.

7.0 Processes

A process is a running instance of a program. Each process has a process identifier (PID), a parent process, credentials, memory mappings, open resources and scheduling state. Administrators monitor processes to understand what the system is doing and which workloads are consuming resources.

7.1 Process Attributes

- PID – unique process identifier.
- PPID – parent process identifier.
- UID/GID – security identity under which the process executes.
- CPU usage – processor time consumed by the process.
- Memory usage – resident and virtual memory associated with the process.
- State – running, sleeping, stopped, zombie or other operating-system state.
- Command – executable or command line that created the process.

```
$ ps -ef | head -n 20
$ top
```

ps -ef gives a snapshot of processes in full-format listing. **top** gives a continuously updating view of system load, CPU, memory and process activity.

In practice, an administrator often starts with top to identify abnormal resource use, then uses more focused commands and logs to validate the cause.

8.0 Services

A service is a long-running capability provided by the system, often running in the background. Examples include secure remote access, web serving, time synchronization, logging and database services.

On many modern Linux distributions, systemd manages service units. Week 2 introduces the concept so that students can connect resource usage to the services that depend on those resources; detailed service administration is developed further in Week 3.

Aspect	Process	Service
Definition	Running instance of a program	Long-running system capability, commonly backed by one or more processes
Lifetime	May be short- or long-lived	Usually designed to remain available
Examples	cp, python, browser process	sshd, web server, database daemon
Management focus	Resource use and execution state	Availability, startup, dependencies and reliability

9.0 Resource Bottlenecks and Evidence-Based Diagnosis

A bottleneck is the resource that currently limits system performance. The same user-visible symptom—such as “the system is slow”—can be caused by different resource constraints. An administrator should therefore avoid guesswork and collect evidence across CPU, memory and storage before deciding on a corrective action.

Symptom	Likely resource	Evidence to collect	Useful Week 2 tools	Possible next step
System slow during computation	CPU	CPU utilisation, load, top consumers	lscpu, nproc, top, ps	Identify CPU-heavy process; validate expected workload.
Applications freeze when many programs open	Memory	Available RAM, swap use, memory-heavy processes	free -h, /proc/meminfo, swapon --show, top	Reduce workload, investigate memory leak, consider capacity.
Writes fail / logs cannot grow	Storage capacity	File-system utilisation and mount locations	df -hT, lsblk -f, findmnt	Identify full filesystem; clean, rotate or expand safely.
Boot/service issues after storage change	Mount/file-system configuration	Block devices, filesystem type, mount mapping	lsblk -f, findmnt	Validate intended mount source and target.
One program consumes excessive resources	Process	PID, owner, CPU and memory use	ps -ef, top	Investigate process purpose, logs and safe remediation.

10.0 Week 2 Individual Practical: Inspecting Linux System Resources

Objective: Inspect the CPU, memory, storage, file systems and processes of a Linux virtual machine, then interpret the evidence in administrator-oriented language. Students should capture outputs or screenshots where required by the lecturer and record a short explanation for each command.

10.1 Practical Preparation

1. Use a Linux virtual machine or approved Linux environment.
2. Open a terminal with a standard user account.
3. Create a working folder for evidence, for example ~/BSIT401/week2.
4. Run commands exactly as shown first; then repeat selected commands after creating a small controlled workload if instructed.
5. Do not terminate unfamiliar system processes or change storage configuration during this inspection exercise.

Task	Command	Evidence / interpretation required
1. CPU architecture	lscpu	Record architecture, CPU count, sockets, cores and virtualisation information.
2. Available processing units	nproc	Compare with the CPU count reported by lscpu.
3. Memory summary	free -h	Record total, available and swap values.
4. Kernel memory details	cat /proc/meminfo | head	Identify important memory fields and relate them to free -h.
5. Swap configuration	swapon --show	Determine whether swap is enabled and where it is located.
6. Block devices and file systems	lsblk -f	Identify disks/partitions, file-system types and mount points.
7. File-system utilisation	df -hT	Identify the root filesystem and the most heavily used mounted filesystem.
8. Mount hierarchy	findmnt	Confirm the source and target for the root filesystem.
9. Process snapshot	ps -ef | head -n 20	Observe PID, PPID, user and command fields.
10. Live resource monitoring	top	Observe load, CPU, memory and the most active processes; exit with q.

10.2 Practical Evidence Template

Command	Key output observed	What it means	Normal/Concern	Follow-up evidence

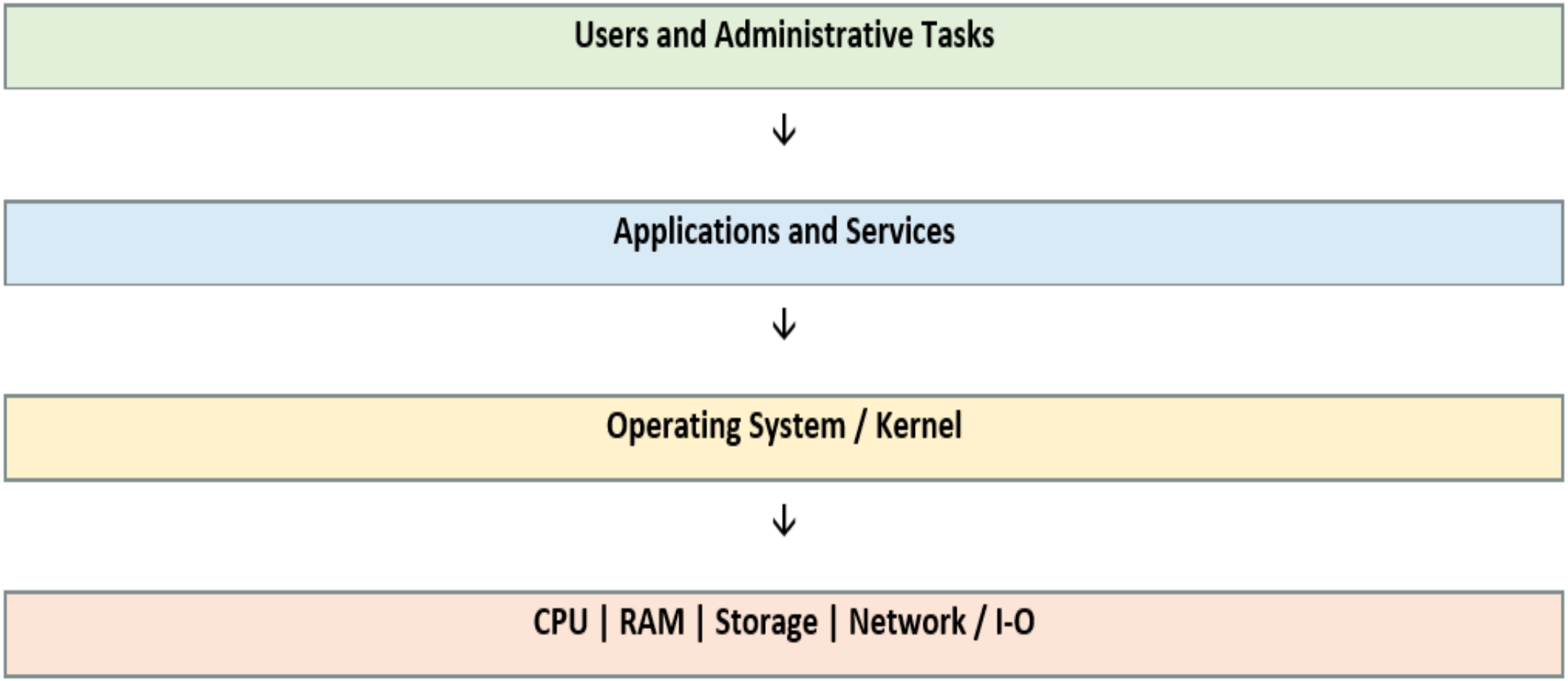
11.0 Worked Administrative Examples

11.1 Example A: CPU Pressure

Scenario: Users report that an analytics server becomes very slow during report generation. The VM has 2 logical CPUs. In top, CPU utilization stays near 100% and one process consistently consumes most CPU time.

Reasoning: The evidence suggests CPU contention. Because the workload is computation-heavy and the CPU is saturated, the administrator should identify the process, confirm whether the high utilization is expected, check whether concurrent jobs can be scheduled differently, and consider additional CPU capacity only after validating workload behavior.

11.1 Layered View of a Computing System



Administrative idea: begin with the user-visible symptom, then move downward through services and the operating system to the underlying resource evidence.

11.2 Storage Relationship Diagram

Physical Disk	→	Partition / Volume	→	File System → Mount Point

Example

/dev/sda (disk) → /dev/sda2 (partition) → ext4 (file system) → / (mount point). A capacity problem must therefore be diagnosed at the correct file-system/mount level, not merely by quoting total disk size.

11.3 CPU–Memory–Storage Performance Relationship

CPU	RAM	Storage
Executes instructions and schedules work	Holds active code/data and cache	Persists OS, applications, logs and user data

Performance diagnosis rule: high CPU utilisation, memory pressure and a nearly full file system can all produce the same user complaint—"the system is slow." The administrator must collect evidence before deciding which resource is the actual bottleneck.

11.2 Example B: Memory Pressure

Scenario: A server with 4 GB RAM becomes sluggish after several applications run for hours. free -h shows very low available memory and swap usage has increased substantially. top shows one application with unusually high memory consumption.

Reasoning: The combined evidence points to memory pressure rather than merely "low free memory." The administrator should investigate the memory-heavy process, determine whether growth is expected or represents a leak, and verify whether the service can be safely restarted or tuned. Capacity expansion is a later decision, not the first diagnostic step.

11.3 Example C: File System Full

Scenario: An application reports "No space left on device." The server has a large secondary disk, but df -hT shows that the root file system is at 100% while the secondary filesystem has ample space.

Reasoning: The problem is not total installed storage; it is exhaustion of the specific mounted filesystem used by the application. The administrator should locate what is consuming space on that filesystem, apply safe cleanup or retention procedures, and consider expansion or relocation if growth is legitimate.

12.0 Group Activity: Diagnosing a Resource Problem

Each group receives a short scenario involving CPU, memory or storage.

The task is to identify the likely cause, state what evidence is needed, and explain how the evidence would support or reject the hypothesis.

The emphasis is on disciplined diagnosis rather than immediately proposing a fix.

Stage	Group question	Expected output	Example evidence
1. Scope	Who/what is affected?	Clearly bounded problem statement	One VM, one service, all users, one filesystem
2. Symptom	What exactly is wrong?	Observable symptom, not assumption	Slow response, OOM, full filesystem
3. Hypothesis	What resource could explain it?	CPU, memory or storage hypothesis	CPU saturation
4. Evidence	What should be collected?	Specific commands and fields	top, free -h, df -hT
5. Interpretation	What result supports/rejects the hypothesis?	Evidence-based conclusion	CPU 100% sustained vs idle CPU
6. Next action	What is the safest next step?	Controlled remediation or deeper investigation	Tune workload, clean storage, inspect process

13.0 Linux-to-Windows Administrative Equivalents

The course uses Linux as the main administration environment, but administrators often work in mixed environments. The following comparisons help students connect the concepts across platforms; the exact output and terminology differ.

Administrative task	Linux example	Windows example
CPU/OS information	lscpu; uname-related tools	Task Manager; System Information; PowerShell Get-CimInstance
Memory utilisation	free -h; /proc/meminfo	Task Manager; Resource Monitor; Get-Counter
Storage/volumes	lsblk -f; df -hT; findmnt	Disk Management; Get-Disk; Get-Volume
Processes	ps; top	Task Manager; Get-Process
Services	systemd tools (developed in Week 3)	Services console; Get-Service

12.0 Expanded Step-by-Step Practical Laboratory Guide

This practical guide translates the Week 2 commands in the course outline into a repeatable administrative workflow. Students should use a Linux virtual machine or approved Linux environment. Run commands as a normal user unless the lecturer specifically authorises elevated privileges.

Recommended evidence folder

Create a dedicated folder for this week so that screenshots, command-output text files and short interpretations can be submitted together. Example: ~/BSIT401/week2

12.1 Practical Exercise 1: Prepare the Linux Environment and Evidence Folder

Purpose

Confirm that you are working in the intended Linux environment and create a folder for Week 2 evidence.

Command(s) to run:

```
hostnamectl
uname -a
mkdir -p ~/BSIT401/week2
cd ~/BSIT401/week2
pwd
```

Step-by-step procedure:

1. Open Terminal.
2. Run hostnamectl and identify the host name, operating system and kernel-related information.
3. Run uname -a to obtain kernel and architecture information.
4. Create the evidence directory using mkdir -p ~/BSIT401/week2.
5. Move into the directory with cd and confirm its path with pwd.
6. Save or capture the relevant output.

Evidence to record:

- Host name and OS distribution/version.
- Kernel version and machine architecture.
- Screenshot showing the Week 2 evidence path.

How to interpret the result:

- The OS and kernel details establish the environment in which later results were collected.
- Architecture information helps explain whether the platform is x86-64, ARM64 or another architecture.

12.2 Practical Exercise 2: Inspect CPU Architecture with lscpu and nproc

Purpose

Determine how Linux sees the processor topology and the number of processing units available to the current environment.

Command(s) to run:

```
lscpu
nproc
```

Step-by-step procedure:

16. Run lscpu.
17. Locate the Architecture field.
18. Locate CPU(s), Core(s) per socket, Socket(s), Thread(s) per core, Model name and Virtualization/Hypervisor fields where present.
19. Run nproc.
20. Compare the nproc result with the CPU(s) value from lscpu.
21. Write a two- to four-sentence interpretation.

Evidence to record:

- Architecture.
- Logical CPU count.
- Sockets and cores.
- Threads per core.
- Processor model.
- Any virtualisation or hypervisor indication.
- nproc result.

How to interpret the result:

- CPU(s) is commonly the number of logical processors visible to the OS.
- nproc reports processing units available to the current process and can differ under containers or CPU restrictions.
- A virtual machine may report virtual CPUs rather than the host computer's full physical CPU topology.

12.3 Practical Exercise 3: Inspect Physical Memory with free -h

Purpose

Measure total, used, available and cached memory using human-readable units.

Command(s) to run:

```
free -h
```

Step-by-step procedure:

22. Run free -h.
23. Read the Mem row and identify total, used, free, buff/cache and available.
24. Read the Swap row.
25. Do not treat the free column alone as the amount of memory that can still be used.
26. Calculate approximate percentage of memory currently used: used ÷ total × 100.
27. Record a short conclusion on whether there is obvious memory pressure.

Evidence to record:

- Total RAM.
- Used RAM.
- Available RAM.
- buff/cache.
- Swap total and used.

How to interpret the result:

- Available memory is generally more useful than “free” for a quick capacity judgement.
- Linux intentionally uses otherwise idle RAM for caching.
- Heavy or increasing swap usage can support a memory-pressure diagnosis, but should be interpreted with workload evidence.

12.4 Practical Exercise 4: Inspect Detailed Memory Information and Swap

Purpose

Use /proc/meminfo and swapon to obtain detailed memory and swap evidence.

Command(s) to run:

```
cat /proc/meminfo | head
swapon --show
```

Step-by-step procedure:

28. Run cat /proc/meminfo | head.
29. Identify MemTotal, MemFree, MemAvailable, Buffers and Cached if shown in the first lines.
30. Run swapon --show.
31. If swap is configured, note its type, size and usage.
32. Compare MemAvailable with the “available” value from free -h.

Evidence to record:

- Relevant /proc/meminfo fields.
- Swap device/file name, type, size and used amount.

How to interpret the result:

- The /proc pseudo-filesystem exposes kernel-maintained system information.
- swapon --show reports configured active swap; an empty result can mean no active swap.

Safety / administrative caution

Do not create, resize or disable swap during this Week 2 inspection unless the lecturer specifically instructs you to do so.

12.5 Practical Exercise 5: Inspect Disks, Partitions and File Systems

Purpose
Identify storage devices, partitions, file-system types, UUIDs and mount points.

Command(s) to run:

```
lsblk -f
```

Step-by-step procedure:

33. Run lsblk -f.
34. Identify the main disk device(s).
35. Identify child partitions or logical volumes.
36. Locate the file-system type (FSTYPE) and mount point(s).
37. Find the device/volume that contains the root file system (/).
38. Record any unmounted partition shown in the output.

Evidence to record:

- Device names.
- Partition/volume relationships.
- File-system types.
- Root (/) source device.
- Other mount points.

How to interpret the result:

- The tree structure shows parent block devices and their children.
- A file system can exist on a partition or logical volume, and it becomes available to users only when mounted.

Safety / administrative caution
Do not format, repartition or mount unfamiliar devices in this exercise. The task is inspection, not destructive storage administration.

12.6 Practical Exercise 6: Check File-System Capacity with df -hT

Purpose
Determine how much space is used and available on mounted file systems.

Command(s) to run:

```
df -hT
```

Step-by-step procedure:

39. Run df -hT.
40. Find the row whose mount point is /.
41. Record its file-system type, total size, used space, available space and Use%.
42. Identify the mounted file system with the highest Use%.
43. Decide whether any file system appears close to a capacity threshold such as 80%, 90% or higher.
44. Explain why a full root file system can affect services even if another disk has free space.

Evidence to record:

- Root file-system Use%.
- Highest Use% among mounted file systems.
- File-system type and mount point for each notable entry.

How to interpret the result:

- df measures mounted file-system capacity, not simply raw physical-disk capacity.
- A high Use% on / or /var can cause application, logging or package-management failures depending on the system layout.

12.7 Practical Exercise 7: Understand Mount Relationships with findmnt

Purpose
Map source devices/file systems to the directories where they are mounted.

Command(s) to run:

```
findmnt
```

Step-by-step procedure:

45. Run findmnt.
46. Locate the / target.
47. Identify its SOURCE and FSTYPE.
48. Locate /proc, /sys and /dev and note that some entries are virtual or pseudo file systems rather than physical disks.
49. Compare the root entry with lsblk -f.

Evidence to record:

- Source and file-system type for /.
- At least two pseudo/virtual file systems.
- One sentence comparing findmnt with lsblk -f.

How to interpret the result:

- findmnt emphasises the mount tree and current attachment points.
- lsblk emphasises block-device relationships. The two tools complement each other.

12.8 Practical Exercise 8: Inspect Running Processes with ps -ef

Purpose

Obtain a static snapshot of running processes and identify process identifiers and ownership.

Command(s) to run:

```
ps -ef | head -n 20
```

Step-by-step procedure:

50. Run ps -ef | head -n 20.
51. Identify the column names.
52. Locate PID and PPID for at least three entries.
53. Identify the user account that owns each selected process.
54. Observe the CMD column and describe the executable or command.
55. Explain why PID 1 is administratively important on a typical systemd-based Linux system.

Evidence to record:

- Three process rows with USER, PID, PPID and CMD.
- A statement about PID 1.

How to interpret the result:

- PID uniquely identifies a process at a given time.
- PPID links a process to its parent.
- The owner indicates the security context in which the process is running.

12.9 Practical Exercise 9: Monitor Resources Interactively with top

Purpose
Observe changing CPU, memory and process activity in real time.

Command(s) to run:

```
top
```

Step-by-step procedure:

56. Run top.
57. Observe uptime and load averages.
58. Observe the CPU summary line(s).
59. Observe memory and swap summaries.
60. Identify the first few processes and their %CPU and %MEM values.
61. Press q to exit.
62. Record the highest CPU and memory consumers you observed.

Evidence to record:

- Load averages.
- Approximate CPU utilisation.
- Available/used memory.
- Top CPU-consuming process.
- Top memory-consuming process.

How to interpret the result:

- A single top snapshot is not proof of a persistent problem; observe trends and repeat if necessary.
- High CPU can be legitimate during intended computation.
- The next administrative question is whether the resource use matches the expected workload.

12.10 Practical Exercise 10: Integrated Resource Diagnosis

Purpose
Use multiple Week 2 commands to diagnose a supplied “slow system” case without guessing.

Command(s) to run:

```
uptime
top
free -h
swapon --show
df -hT
ps -ef | head -n 20
```

Step-by-step procedure:

63. Read the symptom statement provided by the lecturer.
64. Check overall workload with uptime/top.
65. Check memory availability and swap with free -h and swapon --show.
66. Check file-system capacity with df -hT.
67. Inspect processes with ps -ef and top.
68. State one primary hypothesis (CPU, memory, storage or another cause).
69. Quote at least three pieces of command evidence supporting or weakening your hypothesis.
70. Recommend a safe next diagnostic step or corrective action.
71. Explain how you would verify that the problem is resolved.

Evidence to record:

- Symptom.
- Primary hypothesis.
- At least three evidence points.
- Proposed corrective action.
- Verification step.

How to interpret the result:

- Good administration separates observation from hypothesis.
- A diagnosis is stronger when several independent pieces of evidence agree.
- Verification is required after a change; “the command completed” is not the same as “the service is restored.”

12.11 Practical Evidence Recording Template

Command	Key output observed	Interpretation	Potential issue?	Evidence file/screenshot
lscpu				
nproc				
free -h				
cat /proc/meminfo | head				
swapon --show				
lsblk -f				
df -hT				
findmnt				
ps -ef | head -n 20				
top				

12.12 Suggested 3-Hour Practical Delivery Sequence

Time	Activity	Expected output
0-20 min	Concept recap and safety briefing	Students can distinguish CPU, RAM, disk, partition, file system, process and service.
20-55 min	CPU + memory practicals	Completed CPU/memory evidence.
55-95 min	Storage + file-system practicals	Completed storage/mount/capacity evidence.
95-125 min	Processes + top	Process and resource-monitoring evidence.
125-160 min	Integrated diagnosis group case	Evidence-based hypothesis and proposed action.
160-180 min	Debrief and formative quiz	Corrected interpretations and submitted evidence checklist.

13.0 Group Troubleshooting Activity

Each group receives one resource problem involving CPU, memory or storage. The objective is not merely to name the likely fault but to explain how the group would prove or disprove it using Week 2 evidence.

13.1 CPU scenario

Problem statement: A data-analysis workstation becomes extremely slow whenever a scheduled analytics job starts. Users can still access files, and disk space is adequate.

Expected evidence strategy: Use top/ps to identify high CPU processes; compare load with CPU count from nproc/lscpu; determine whether utilisation corresponds to the scheduled job.

13.2 Memory scenario

Problem statement: A VM becomes sluggish after several applications have been running for hours. The system remains responsive but begins using swap.

Expected evidence strategy: Use free -h, /proc/meminfo, swapon --show and top; distinguish low “free” RAM from genuinely low “available” memory and identify memory-heavy processes.

13.3 Storage scenario

Problem statement: An application reports “No space left on device,” although another attached disk has hundreds of gigabytes free.

Expected evidence strategy: Use df -hT to locate the full mounted file system; use lsblk -f and findmnt to map it to the correct volume/mount point; explain why free space elsewhere does not solve the affected mount.

14.0 Windows Administrative Equivalents (Comparative Awareness)

Linux Week 2 goal	Linux tool	Windows equivalent / GUI path
CPU details	lscpu, nproc	Task Manager > Performance > CPU; PowerShell Get-CimInstance Win32_Processor
Memory status	free -h	Task Manager > Performance > Memory; Get-CimInstance Win32_OperatingSystem
Disks/volumes	lsblk -f, df -hT	Disk Management; Get-Disk; Get-Volume
Processes	ps -ef, top	Task Manager > Processes/Details; Get-Process
Services concept	system services	Services.msc; Get-Service

14.0 Administrative Best Practices for Resource Inspection

- Collect evidence before changing configuration.
- Establish a baseline when the system is healthy so abnormal behaviour can be recognized later.
- Use more than one metric to validate a suspected bottleneck.
- Interpret resource usage in context: workload, time, number of CPUs, service importance and expected demand.
- Avoid terminating processes simply because they use high CPU or memory; determine whether the usage is legitimate.
- Distinguish total device capacity from the capacity of the mounted filesystem that actually contains the data.
- Record command, timestamp, observed values and interpretation when troubleshooting.
- Prefer reversible and low-risk corrective actions, and verify the result after any change.

15.0 Review and Formative Assessment Questions

15.1 Short-Answer Questions

1. Explain why low “free” memory does not always mean a Linux system is short of memory.

2. Differentiate a physical disk, partition, file system and mount point.

3. What administrative information can be obtained from lscpu?

4. Why might nproc report a number that differs from the total CPU count of a host?

5. What is swap and under what circumstances can increasing swap usage be a warning sign?

6. How does df -hT differ from lsblk -f?

7. What is a PID and why is it important to an administrator?

8. Differentiate a process from a service.

9. Why should an administrator collect evidence before restarting a service or killing a process?

10. What does it mean to say that a resource is a bottleneck?

15.2 Multiple-Choice Questions

1. Which command most directly displays Linux block devices together with file-system information?

A. free -h
B. lsblk -f
C. ps -ef
D. nproc

2. Which value from free -h is generally the best quick indicator of how much memory can be used for new work without heavy swapping?

A. free
B. available
C. swap total
D. shared

3. Which command provides a continuously updating view of CPU, memory and processes?

A. top
B. findmnt
C. df -hT
D. swapon --show

4. A server has a 2 TB disk but df -hT shows / is 100% full. What is the most accurate conclusion?

A. The entire 2 TB disk must be full.
B. The root filesystem is full; other filesystems may still have space.
C. CPU is saturated.
D. Swap is disabled.

5. What is the main administrative purpose of findmnt?

A. Show CPU topology
B. Show mounted filesystems and their hierarchy
C. Show running processes
D. Show only swap usage

15.3 Applied Scenario Questions

1. A user says, “The server is slow.” Write a six-step evidence collection plan using only the Week 2 resource categories and commands.
2. A Linux VM has 4 logical CPUs and a load average that remains around 10 while CPU utilisation is close to 100%. Explain what this could indicate and what evidence you would collect next.
3. An application cannot create new log files. df -hT shows /var on a separate filesystem at 100%, while / has 40% free. Explain the cause and identify the correct administrative focus.
4. free -h reports little free RAM but several gigabytes available and no swap activity. Is this necessarily a memory problem? Justify your answer.

18.0 End-of-Week Competency Checklist

☐ I can explain CPU sockets, cores and logical processors.

☐ I can use lscpu and nproc and interpret their main outputs.

☐ I can explain RAM, cache, available memory and swap.

☐ I can use free -h, /proc/meminfo and swapon --show.

☐ I can distinguish disks, partitions, file systems and mount points.

☐ I can use lsblk -f, df -hT and findmnt.

☐ I can explain process identifiers and use ps -ef and top.

☐ I can explain the relationship between processes and services.

☐ I can diagnose a basic CPU, memory or storage scenario using evidence.

☐ I can document the commands, observations and administrative conclusion clearly.

17.0 Multiple-Choice Questions (MCQs)

1. Which command provides a structured summary of Linux CPU architecture?

- A. df -hT
- B. lscpu
- C. findmnt
- D. swapon --show

2. Which command reports the number of processing units available to the current process?

- A. nproc
- B. ps
- C. free
- D. lsblk

3. Which free -h column is generally most useful for estimating memory headroom?

- A. total
- B. shared
- C. available
- D. buff/cache only

4. What does swap primarily provide?

- A. Permanent file storage
- B. Disk-backed support for virtual memory
- C. CPU cache
- D. Network buffering only

5. Which command is most appropriate for viewing active swap devices or files?

- A. swapon --show
- B. df -hT
- C. nproc
- D. ps -ef

6. Which command shows block devices together with file-system information?

- A. lscpu
- B. lsblk -f
- C. top
- D. hostnamectl

7. Which command is best for checking utilisation of mounted file systems?

- A. df -hT
- B. lscpu
- C. nproc
- D. uname -a

8. Which tool focuses on the active mount hierarchy?

- A. findmnt
- B. free -h
- C. ps -ef
- D. nproc

9. A mount point is best described as:

- A. A CPU register
- B. A directory where a file system becomes accessible
- C. A process identifier
- D. A swap algorithm

10. Which is a common Linux file system?

- A. ext4
- B. HTTP
- C. DHCP
- D. SMTP

11. A PID identifies:

- A. A partition
- B. A file system
- C. A process
- D. A CPU cache

12. PPID refers to:

- A. Parent Process Identifier
- B. Physical Partition ID
- C. Processor Performance ID
- D. Primary Package Installation Directory

13. Which command gives a full-format snapshot of processes?

- A. ps -ef
- B. df -hT
- C. free -h
- D. findmnt

14. Which command provides an interactive, continuously refreshed resource view?

- A. lsblk
- B. top
- C. nproc
- D. cat

15. A service is best described as:

- A. Always a hardware device
- B. A long-running capability commonly backed by one or more processes
- C. A disk partition
- D. A CPU core

16. Low “free” memory by itself means:

- A. The system definitely needs more RAM
- B. The CPU is overloaded
- C. Nothing conclusive without considering available memory and other evidence
- D. The file system is corrupted

17. A file system at 99% utilisation should primarily be investigated with:

- A. df -hT and findmnt
- B. nproc and lscpu
- C. uname and hostnamectl
- D. swapon only

18. If an 8-core host gives a VM 2 vCPUs, lscpu inside the VM will normally describe:

- A. The full physical host only
- B. The VM-visible CPU resources
- C. Disk capacity
- D. Network bandwidth

19. Which statement about evidence-based troubleshooting is most accurate?

- A. Apply a fix before collecting evidence
- B. One symptom always has one cause
- C. Form and test a hypothesis using multiple pieces of evidence
- D. Rebooting is always the first step

20. Which item is volatile?

- A. RAM
- B. SSD
- C. HDD
- D. Optical disk

21. Which storage technology normally provides the lowest latency among the choices?

- A. HDD
- B. NVMe SSD
- C. Magnetic tape
- D. DVD

22. Which command can reveal the root file system's source and type?

- A. findmnt
- B. nproc
- C. lscpu
- D. whoami

23. Which statement correctly distinguishes raw disk capacity from file-system capacity?

- A. They are always identical
- B. A disk can contain multiple partitions/file systems, so mounted capacity must be checked separately
- C. File systems exist only in RAM
- D. Partitions cannot be mounted

24. Which metric in top should be treated as a time-varying observation rather than a permanent fact?

- A. Current %CPU of a process
- B. CPU architecture
- C. Disk UUID
- D. File-system type

25. A process consuming high CPU should be terminated immediately when detected. This statement is:

- A. Always true
- B. False; first determine whether the workload is expected and collect evidence
- C. True only on Linux
- D. True if the process is owned by root

26. Which command exposes detailed kernel memory statistics through a pseudo-file?

- A. cat /proc/meminfo
- B. df -hT
- C. nproc
- D. findmnt

27. Which is the strongest evidence of a storage-capacity problem?

- A. A large disk model name
- B. A mounted file system at or near 100% utilisation correlated with write failures
- C. Many CPU cores
- D. A low PPID

28. Why should students compare lsblk -f and findmnt?

- A. They are identical commands
- B. One shows device relationships while the other emphasises active mounts
- C. Both calculate CPU load
- D. Both manage passwords

29. The best final step after correcting a resource-related fault is to:

- A. Assume success
- B. Verify that the symptom/service has returned to normal using evidence
- C. Delete the logs
- D. Shut down the server

30. Which set contains only Week 2 prescribed inspection commands from the course outline?

- A. lscpu, free -h, lsblk -f, df -hT, ps -ef
- B. useradd, chmod, ufw, git, ansible
- C. traceroute, ping, resolvectl, ip route only
- D. docker, terraform, git, ssh-keygen

End of Lectures

Thank you